

Task 5

1. Executive Summary & Deliverables

Task 5 provides the core security and identity foundation for the DDoS mobile platform[span_4](start_span)[span_4](end_span). It exposes secure endpoints for user registration, authentication, JWT session token generation, and authorization middleware to protect private API routes across all downstream backend modules[span_5](start_span)[span_5](end_span).

Key Technical Deliverables:

- * **POST /auth/register**: Hashes user passwords using Bcrypt (10 salt rounds) and creates new user accounts in PostgreSQL[span_6](start_span)[span_6](end_span).

- * **POST /auth/login**: Validates user credentials and issues signed JsonWebTokens (JWT) valid for 7 days[span_7](start_span)[span_7](end_span).

- * **auth.middleware.ts**: Express middleware validating `Authorization: Bearer <token>` headers on protected routes[span_8](start_span)[span_8](end_span).

- * **adminOnly.middleware.ts**: Role-Based Access Control (RBAC) ensuring administrative routes are restricted to admin users[span_9](start_span)[span_9](end_span).

2. Directory Structure

```text

```
src/
├── routes/
│ └── auth.routes.ts # Express router mapping /auth endpoints
├── controllers/
│ └── auth.controller.ts # Input validation & authentication business logic
├── middleware/
│ ├── auth.middleware.ts # JWT session token verification middleware
│ └── adminOnly.middleware.ts # Role-Based Access Control (RBAC) guard
└── utils/
 ├── jwt.ts # Token generation and verification helpers
 └── prisma.ts # Prisma ORM database client setup
```

### 3. API Data Contracts

#### 3.1 User Registration (POST /auth/register)

Request Payload:

```
{
 "name": "Jeevana Jyothi",
 "email": "chellajeevanajyothi@gmail.com",
 "password": "SecurePassword123!"
}
```

Response Payload (201 Created):

```
{
 "id": "c3a9f810-7e12-4c6d-9b0a-42a1b9812a02",
 "email": "chellajeevanajyothi@gmail.com",
 "name": "Jeevana Jyothi",
 "role": "USER",
 "createdAt": "2026-08-14T10:25:00.000Z"
}
```

### 3.2 User Login (POST /auth/login)

Request Payload:

```
{
 "email": "chellajeevanajyothi@gmail.com",
 "password": "SecurePassword123!"
}
```

Response Payload (200 OK):

```
{
 "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "user": {
 "id": "c3a9f810-7e12-4c6d-9b0a-42a1b9812a02",
 "email": "chellajeevanajyothi@gmail.com",
 "name": "Jeevana Jyothi",
 "role": "USER"
 }
}
```

## 4. TypeScript Source Code

### 4.1 Auth Controller (src/controllers/auth.controller.ts)

```
import { Request, Response } from 'express';
import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';
import { prisma } from '../utils/prisma';
```

```
export const register = async (req: Request, res: Response): Promise<Response> => {
 try {
 const { email, password, name } = req.body;

 if (!email || !password || password.length < 6) {
 return res.status(400).json({ error: 'Valid email and password (min 6 chars) required.' });
 }

 const existingUser = await prisma.user.findUnique({ where: { email } });
 if (existingUser) {
 return res.status(400).json({ error: 'User already exists with this email.' });
 }

 const hashedPassword = await bcrypt.hash(password, 10);
 const user = await prisma.user.create({
 data: {
```

```

 email,
 password: hashedPassword,
 name,
 role: 'USER',
 },
});

return res.status(201).json({
 id: user.id,
 email: user.email,
 name: user.name,
 role: user.role,
});
} catch (error) {
 return res.status(500).json({ error: 'Internal server error during registration.' });
}
};

export const login = async (req: Request, res: Response): Promise<Response> => {
 try {
 const { email, password } = req.body;

 const user = await prisma.user.findUnique({ where: { email } });
 if (!user || !(await bcrypt.compare(password, user.password))) {
 return res.status(401).json({ error: 'Invalid email or password.' });
 }

 const token = jwt.sign(
 { id: user.id, role: user.role },
 process.env.JWT_SECRET || 'super_secret_jwt_key',
 { expiresIn: '7d' }
);

 return res.json({
 token,
 user: {
 id: user.id,
 email: user.email,
 name: user.name,
 role: user.role,
 },
 });
 } catch (error) {
 return res.status(500).json({ error: 'Internal server error during login.' });
 }
};

```

#### 4.2 Auth Middleware (src/middleware/auth.middleware.ts)

```
import { Request, Response, NextFunction } from 'express';
```

```

import jwt from 'jsonwebtoken';

export interface AuthenticatedRequest extends Request {
 user?: {
 id: string;
 role: string;
 };
}

export const authMiddleware = (
 req: AuthenticatedRequest,
 res: Response,
 next: NextFunction
) => {
 const token = req.headers.authorization?.split(' ')[1];

 if (!token) {
 return res.status(401).json({ error: 'Access denied. Authorization token required.' });
 }

 try {
 const decoded = jwt.verify(
 token,
 process.env.JWT_SECRET || 'super_secret_jwt_key'
) as { id: string; role: string };

 req.user = decoded;
 next();
 } catch {
 return res.status(401).json({ error: 'Invalid or expired session token.' });
 }
};

export const adminOnly = (
 req: AuthenticatedRequest,
 res: Response,
 next: NextFunction
) => {
 if (!req.user || req.user.role !== 'ADMIN') {
 return res.status(403).json({ error: 'Forbidden: Requires administrative privileges.' });
 }
 next();
};

```